P.D.E.A's
Prof. Ramkrishna More A.C.S. College (Autonomous) Akurdi,Pune-44
Department Of Computer Science
T.Y.B.Sc(CS)
CSMAP-354 Operating System and Data Science Laboratory
Assignment 2: Statistical Data Analysis

Set-A

Qu.1 Write a Python program to find the maximum and minimum value of a given flattened array.
Expected Output:
Original flattened array:
[[0 1]
[2 3]]
Maximum value of the above flattened array:3
Minimum value of the above flattened array:0

```
import numpy as np

# Original 2D array
array = np.array([[0, 1], [2, 3]])
print("Original flattened array:")
print(array)

# Flatten the array
flat_array = array.flatten()

# Find the maximum and minimum values
max_value = np.max(flat_array)
min_value = np.min(flat_array)

print("\nMaximum value of the above flattened array:")
print(max_value)
print("Minimum value of the above flattened array:")
print(min_value)
```

Qu.2 Write a python program to compute Euclidian Distance between two data points in a dataset. [Hint: Use linalgo.norm function from NumPy]

```
import numpy as np

# Define two data points
point1 = np.array([1, 2, 3])
point2 = np.array([4, 5, 6])
```

```
# Compute Euclidean distance
distance = np.linalg.norm(point1 - point2)
print("The Euclidean distance between {point1} and {point2} is {distance}")
```

Qu.3 Create one dataframe of data values. Find out mean, range and IQR for this data.

```
import pandas as pd

# Create a DataFrame
data = {
    'A': [10, 20, 30, 40, 50],
    'B': [15, 25, 35, 45, 55],
    'C': [5, 10, 15, 20, 25]
}
df = pd.DataFrame(data)
print("Original DataFrame:")
print(df)

# Calculate mean for each column
mean_values = df.mean()
print("\nMean values:")
print(mean_values)

# Calculate range (max - min) for each column
range_values = df.max() - df.min()
print("\nRange values:")
print(range_values)

# Calculate IQR (Interquartile Range) for each column
iqr_values = df.quantile(0.75) - df.quantile(0.25)
print("\nIQR values:")
print(iqr_values)
```

Qu.4 Write a python program to compute sum of Manhattan distance between all pairs of points.

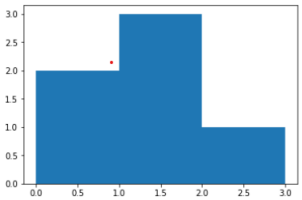
```
import numpy as np
from itertools import combinations

# Example dataset of points
points = np.array([[1, 2], [3, 4], [5, 6]])

# Compute the sum of Manhattan distances between all pairs of points
sum_manhattan_distance = sum(
    np.sum(np.abs(p1 - p2)) for p1, p2 in combinations(points, 2)
)

print(f"The sum of Manhattan distances between all pairs of points is
{sum_manhattan_distance}")
```

Qu.5 Write a NumPy program to compute the histogram of nums against the bins.
Sample Output:
nums: [0.5 0.7 1. 1.2 1.3 2.1]
bins: [0 1 2 3]
Result: (array([2, 3, 1], dtype=int64), array([0, 1, 2, 3]))




```
import numpy as np
import matplotlib.pyplot as plt

# Sample data
nums = np.array([0.5, 0.7, 1.0, 1.2, 1.3, 2.1])
bins = np.array([0, 1, 2, 3])

# Compute the histogram
hist, bin_edges = np.histogram(nums, bins)
```

```
# Display the results
print("nums:", nums)
print("bins:", bins)
print("Result:", (hist, bin_edges))

# Plot the histogram
plt.hist(nums, bins, edgecolor='black')
plt.xlabel('Bins')
plt.ylabel('Frequency')
plt.title('Histogram of nums against bins')
plt.show()
```

Qu.6 Create a dataframe for students' information such name, graduation percentage and age. Display average age of students, average of graduation percentage. And, also describe all basic statistics of data. (Hint: use describe()).

```
import pandas as pd

# Create DataFrame
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'Graduation Percentage': [85, 90, 78, 92, 88],
    'Age': [22, 23, 21, 24, 22]
}
df = pd.DataFrame(data)

# Calculate averages
average_age = df['Age'].mean()
average_grad_percentage = df['Graduation Percentage'].mean()

# Display results
print(f"Average age: {average_age}")
print(f"Average graduation percentage: {average_grad_percentage}")

# Describe statistics
print(df.describe())
```

P.D.E.A's
Prof. Ramkrishna More A.C.S. College (Autonomous) Akurdi,Pune-44
Department Of Computer Science
T.Y.B.Sc(CS)
CSMAP-354 Operating System and Data Science Laboratory
Assignment 2: Statistical Data Analysis

Set-B

Data download Link: https://archive.ics.uci.edu/dataset/53/iris

Qu.1 Download iris dataset file. Read this csv file using read_csv() function. Take samples from entire dataset. Display maximum and minimum values of all numeric attributes.

```
import pandas as pd
column_names = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'class']
df=pd.read_csv("C:/Users/admin/Desktop/Dataset/iris/iris.data", header=None,
names=column_names)

print(df)
print("Iris Dataset:")
print(df.head())

# Take a sample from the dataset
sample_df = df.sample(frac=0.1)  # Sample 10% of the data
print("\nSample of the Dataset:")
print(sample_df)

# Display maximum and minimum values of all numeric attributes
max_values = df.max(numeric_only=True)
min_values = df.min(numeric_only=True)

print("\nMaximum values of all numeric attributes:")
print(max_values)
print("\nMinimum values of all numeric attributes:")
print(min_values)
```

Qu.2 Continue with above dataset, find number of records for each distinct value of class attribute. Consider entire dataset and not the samples.

```
import pandas as pd
import matplotlib.pyplot as plt
column_names = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'class']
df=pd.read_csv("C:/Users/admin/Desktop/Dataset/iris/iris.data", header=None,
names=column_names)
print(df)
```

```
# Find number of records for each distinct value of class attribute
class_counts = df['class'].value_counts()

print("Number of records for each distinct class:")
print(class_counts)

# Visualize the class distribution
plt.figure(figsize=(8, 6))
class_counts.plot(kind='bar')
plt.title('Number of Records for Each Distinct Class')
plt.xlabel('Class')
plt.ylabel('Count')
plt.tight_layout()
plt.show()
```

Qu 3. Display column-wise mean, and median for iris dataset from Q.4 (Hint: Use mean() and median() functions of pandas dataframe.

```
import pandas as pd
column_names = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'class']
df=pd.read_csv("C:/Users/admin/Desktop/Dataset/iris/iris.data", header=None,
names=column_names)
print(df)
# Calculate and display column-wise mean
print("Column-wise Mean:")
print(df.mean(numeric_only=True))
print("\n")

# Calculate and display column-wise median
print("Column-wise Median:")
print(df.median(numeric_only=True))
```

Set-C

Qu 1. Write a python program to find Minkowskii Distance between two points.

```
import numpy as np
def minkowski_distance(p, q, k):
    p = np.array(p)
    q = np.array(q)
    # Compute the Minkowski distance
    distance = np.sum(np.abs(p - q) ** k) ** (1 / k)
    return distance

# Example usage
p = [1, 2, 3]
q = [4, 5, 6]
k = 3  # Change k to compute different types of Minkowski distances

distance = minkowski_distance(p, q, k)
print(f"The Minkowski distance between {p} and {q} with k={k} is
{distance:.2f}")
```

Qu 2. Write a Python NumPy program to compute the weighted average along the specified axis of a given flattened array. From Wikipedia: The weighted arithmetic mean is similar to an ordinary arithmetic mean (the most common type of average), except that instead of each of the data points contributing equally to the final average, some data points contribute more than others. The notion of weighted mean plays a role in descriptive statistics and also occurs in a more general form in several other areas of mathematics.

Sample output:
Original flattened array:
[[0 1 2]
[3 4 5]
[6 7 8]]
Weighted average along the specified axis of the above flattened array:
[1.2 4.2 7.2]
Sample Output:
Original array1:
[0 1 3]
Original array2:
[2 4 5]
Cross-correlation of the said arrays:

[[2.33333333 2.16666667]
[2.16666667 2.33333333]]

```
import numpy as np
# Define the flattened array and weights
array = np.array([[0, 1, 2],
                  [3, 4, 5],
                  [6, 7, 8]])
weights = np.array([1, 1, 1])  # Example weights for the columns
# Compute the weighted average along axis 0 (columns)
weighted_average = np.average(array, axis=0, weights=weights)
print("Original flattened array:")
print(array)
print("\nWeighted average along the specified axis of the above flattened array:")
print(weighted_average)
```

Qu 3. Write a NumPy program to compute cross-correlation of two given arrays.

```
import numpy as np
def cross_correlation_1d(a, b):
    return np.correlate(a, b, mode='full')
# Define the arrays
array1 = np.array([0, 1, 3])
array2 = np.array([2, 4, 5])
# Compute the cross-correlation
cross_corr = cross_correlation_1d(array1, array2)
print("Original array1:")
print(array1)
print("\nOriginal array2:")
print(array2)
print("\nCross-correlation of the said arrays:")
print(cross_corr)
```

Qu 4 . Create one dataframe with 5 subjects and marks of 10 students for each subject. Find arithmetic mean, geometric mean, and harmonic mean.

```
import pandas as pd
import numpy as np
from scipy.stats import hmean, gmean
```

```
# Create a DataFrame with 5 subjects and marks for 10 students
data = {
    'Core Java': [85, 90, 78, 92, 88, 76, 95, 89, 84, 91],
    'Operating System': [78, 83, 80, 85, 82, 77, 88, 81, 79, 86],
    'Data Science': [88, 85, 91, 89, 87, 90, 92, 84, 88, 86],
    'Web Technology': [92, 88, 84, 90, 86, 91, 89, 87, 85, 93],
    'Web Designing': [79, 81, 77, 83, 80, 82, 78, 84, 79, 85]
}

df = pd.DataFrame(data)

# Compute the arithmetic mean for each subject
arithmetic_mean = df.mean()

# Compute the geometric mean for each subject
geometric_mean = df.apply(lambda x: gmean(x[x > 0]))

# Compute the harmonic mean for each subject
harmonic_mean = df.apply(lambda x: hmean(x[x > 0]))

# Display the results
print("Arithmetic Mean:")
print(arithmetic_mean)
print("\nGeometric Mean:")
print(geometric_mean)
print("\nHarmonic Mean:")
print(harmonic_mean)
```

Self-Study Questions:

1. Download any dataset from UCI (do not repeat the dataset) . Read this csv file using read_csv() function. Describe the dataset using appropriate function. Display mean value of numeric attribute. Check any data values are missing or not.
2. Download nursery dataset from UCI. Split dataset on any one categorical attribute. Compare the means of each split. (Use groupby)
   Download the dataset from :
   https://archive.ics.uci.edu/dataset/76/nursery